How the Circuit Simulator Works (Simple Explanation)

Oct 7, 2026 · @AnujS

This explains, in plain language, everything that happens behind the scenes when someone presses **Run Simulation** on a drawn circuit — how the drawing becomes numbers, how those numbers are calculated, and how they turn into the arrows and labels seen on screen. Wherever a calculation happens in a specific piece of code, this doc names which file it lives in and what that piece of code is responsible for — without using technical file paths or raw function signatures.

1. The Big Picture

Here is the most important thing to understand first: **the app itself does not do any circuit math.** It does not calculate voltages or currents using its own formulas.

Instead, here is what really happens in four big stages:

1. **The drawing becomes a list.** When someone draws resistors, batteries, wires and so on, the app looks at the picture and turns it into a plain list: "this part is connected to that point, that part is connected to this other point," and so on. This list has no electrical meaning yet — it is just a map of connections.
2. **The list becomes a file a solver can read.** That connection list is sent to the server (the backend), which rewrites it into a special text file in a format understood by a separate, well-tested circuit-solving program called **ngspice**. Ngspice has existed for decades and is trusted by real electrical engineers, so instead of reinventing circuit math, the app leans on it.
3. **Ngspice solves the circuit.** Ngspice reads that file and does the actual math — figuring out the voltage at every point and the current flowing through every part. This is the only place in the whole system where real circuit equations are solved.
4. **The answers come back and get dressed up for the screen.** The raw numbers ngspice produces are plain text. The backend organizes them into a clean per-part summary (voltage in, voltage out, current, direction). Then the part of the app running in the browser (the frontend) uses that summary to draw arrows, color changes, number labels, and animations on top of the picture the person drew.

A simple way to remember it: **drawing → connection list → solver file → real math by ngspice → organized results → visuals on screen.**

Two separate programs are involved:

- The **frontend** is the part that runs in the person's browser. It is written in React (a popular tool for building interactive web pages). It handles drawing, turning the drawing into a connection list, and turning results into visuals.
- The **backend** is the part that runs on a server. It is written using FastAPI (a tool for building web services in Python). It takes the connection list, builds the solver file, runs ngspice, and organizes the results.

The table below shows, for each type of result, which side calculates it and roughly where in the code that happens — described by what the code does, not by its exact file path.

What is being figured out	Who figures it out	Roughly where
Which connection points are actually the same electrical point	The frontend, using a grouping technique described in Section 2	A file that builds the connection list, in a function that builds that list
The voltage at each point in the circuit	Ngspice, using real circuit-solving math	The backend's main server file, when it runs ngspice
The current flowing through each component	Ngspice, read out using a clever trick explained in Section 3	The backend's main server file, in the function that prepares the solver file
The voltage difference across each part, which direction current flows, and rounding tiny near-zero values down to exactly zero	The backend, after ngspice has already solved everything	The backend's main server file, in the function that organizes per-part results
The current flowing through each individual wire	The frontend, using a classic electrical rule called Kirchhoff's Current Law (explained in Section 7)	A frontend file dedicated to working out wire currents
The voltage number shown labeled on a wire	The frontend, just by looking up the voltage of the point that wire belongs to	The frontend's main canvas-drawing file

The rest of this document walks through each of these stages slowly, in order.

2. Step 1 — Turning the Drawing Into a Connection List

When someone presses Run Simulation, the first thing that happens lives entirely in the browser (the frontend). A function in the main app-state file — the file responsible for remembering everything about the current circuit — kicks things off. It calls another function, in a file dedicated to building connection lists, that does the actual translation from "drawing" to "list of connections," and then sends that list to the backend over the internet.

2.1 Pins, wires and nets — the basic vocabulary

A few words are used over and over in this system, so it helps to define them plainly:

- A **pin** is a connection point on a component. A resistor has two pins. A battery has two pins (a `+` and a `−`). A ground symbol has one pin.
- A **wire** simply connects exactly two pins together. A wire has no electrical personality of its own — no resistance, no length that matters, nothing. It is purely a statement that "these two points are the same point."
- A **net** (also sometimes called a node) is a whole group of pins that all end up connected to each other, whether directly by one wire or through a chain of several wires. Every pin inside the same net always has exactly the same voltage, because electrically speaking, they are all the same point.

Every kind of component has its pins listed in a fixed, specific order, and this order matters a lot for everything that comes later. The first pin in that order is always called **pin a**, and the second is always called **pin b**. For example:

Kind of component	Pin a (the first pin)	Pin b (the second pin)
Resistor, capacitor, inductor	p1	p2
DC source, AC source, battery	the `+` terminal	the `−` terminal
Dependent source	terminal A	terminal B
Diode	the anode	the cathode
Ground	its single pin	(none)

This pin order is defined once, centrally, in a file that lists every kind of component the app knows how to draw.

2.2 Grouping pins into nets: the "union-find" technique

To figure out which pins belong to the same net, the connection-list-building function uses a classic computer-science technique called **union-find** (sometimes called "disjoint-set"). In plain terms, here is what it does:

1. **Start separate.** Every single pin on every single component starts out in its own private group, all by itself. Even a pin with no wire touching it still gets its own group.
2. **Merge groups along wires.** For every wire drawn, the two pins it connects get merged into the same group.
3. **Each final group is one net.** After going through every wire, whatever pins ended up in the same group are treated as one electrical point.

The nice thing about this approach is that it does not care *how* the wires are arranged on screen — a straight wire, a wire that bends through several points, a T-shaped junction where three wires meet, anything. However the wires are drawn, any pins that are connected end up grouped into the same net.

2.3 Naming the nets, and choosing a ground point

Every circuit needs one reference point that is treated as "zero volts," because all other voltages are measured relative to it. This reference point is always named `"0"`.

- If someone drew a ground symbol, whichever net that ground symbol's pin belongs to is automatically named `"0"`.
- If nobody drew a ground symbol at all, the circuit still needs a reference point to be solvable, so the app quietly picks one: the `-` terminal of the very first voltage source becomes the hidden `"0"` point. If there is not even a voltage source, the second pin of the very first component found becomes the reference instead.
- Every other net is simply given the next available number: `"1"`, then `"2"`, and so on, in the order they are first discovered.
- Ground symbols themselves are never sent along as if they were real components in the list. They exist only to mark which net should be called `"0"`.

2.4 What the finished connection list looks like

Once this is done, the frontend has produced a simple structured list that says, for every component, which net its pin a touches and which net its pin b touches, along with that component's values (like a resistor's resistance, or a battery's voltage). For example, a 10-volt battery connected to two resistors in series might produce a list roughly like:

- Battery, connected between net "1" and net "0", at 10 volts
- First resistor, connected between net "1" and net "2", at 1,000 ohms
- Second resistor, connected between net "2" and net "0", at 4,000 ohms

For every component in this list, the first net listed always corresponds to pin a, and the second always corresponds to pin b — that ordering is never mixed up, and it matters for every calculation that follows later in this document.

3. Step 2 — Turning the Connection List Into a File ngspice Understands

Once the connection list arrives at the backend server, a function dedicated to building the solver's input file — found in the backend's main server file — turns it into a plain text document, often called a "deck." This deck is written in the specific language that ngspice expects.

3.1 Re-checking ground

The backend double-checks which net should be `"0"` on its own, even though the frontend already decided this. It does this so that connection lists coming from somewhere other than the drawing tool — for example, a feature that reads a circuit out of an uploaded image — still work correctly.

3.2 Giving every part a unique name

Each component in the deck gets a unique name made of a letter (indicating its type), a number (so two components never clash even if they happen to share the same label), and its original label. For example, a battery might become something like "V0_V1," and a resistor might become "R1_R1." This matters because two different palettes of components in the app — the DC source palette and the AC source palette — could otherwise both try to create something labeled "V1," which would confuse the solver.

Here is, in plain terms, what each kind of component turns into in the deck:

Component type	What letter it gets	What gets written
Resistor	R	A line giving its resistance in ohms. If no value was set, 1,000 ohms is used by default. A resistance of exactly 0 is treated as an error risk, so it is quietly replaced with 1,000 ohms.
Capacitor	C	A line giving its capacitance in farads. Defaults to 1 microfarad if unset.
Inductor	L	A line giving its inductance in henries. Defaults to 1 millihenry if unset.
Diode	D	A line referencing a standard diode model that behaves like a common real-world diode (the 1N4148).
DC source, battery, dependent source	V	A line giving a fixed voltage.
AC source	V	A line describing a wave that oscillates at a given size and frequency.
Ground	(nothing)	Ground symbols are never written into the deck as components at all — they only influence which net is named "0," as explained in Section 2.3.

3.3 The clever trick for measuring current: "sense" sources

Here is something that is not obvious at first: ngspice can only directly report the current flowing through **voltage sources**. It cannot directly tell you the current through a plain resistor, capacitor, inductor, or diode.

To work around this, the deck-building function inserts a tiny invisible helper for every resistor, capacitor, inductor, and diode: a **zero-volt voltage source**, wired in series right after that component. A zero-volt source changes absolutely nothing about how the circuit behaves electrically — it acts exactly like a plain wire, since it adds no voltage of its

own. But because it *is* technically a voltage source, ngspice is now willing to report the current flowing through it, and that current is exactly the same as the current flowing through the real component right before it.

Think of it like inserting an ammeter (a current-measuring device) in series with a part, except this ammeter has zero resistance and changes nothing about the circuit, and it exists only so the solver has a way to answer the question "how much current is flowing here?"

Real voltage sources (like batteries) do not need this trick — ngspice already reports their current automatically.

The backend keeps track, for every single component in the circuit, of exactly which of these "sense" measurements corresponds to its current, so that later, when organizing results, it knows exactly where to look.

3.4 The instructions at the bottom of the deck

At the very end of the deck, a short block of instructions tells ngspice what to actually do: solve the circuit (either a simple DC calculation, or an AC calculation — explained in Section 9), and then print out the voltage of every net and the current from every voltage source and sense source. The printed numbers are requested with very high precision (12 significant digits) specifically so that later sanity-checks — like confirming that power supplied equals power consumed — come out matching exactly, rather than being thrown off by tiny rounding errors.

The little helper connection points created by the sense-source trick are never printed or shown to the user — only real, named nets and real component currents are.

4. Step 3 — Letting ngspice Solve the Circuit

A function in the backend's main server file writes the deck out to a temporary file, then launches ngspice to process it in "batch mode" (meaning it runs automatically with no human clicking through a program window), and waits up to 30 seconds for an answer. A separate small helper function is responsible for locating the ngspice program itself on the computer running the server, checking a few likely locations one after another until it finds it.

4.1 What ngspice is actually calculating

Ngspice uses a well-established mathematical method for circuit-solving called **Modified Nodal Analysis**. In plain language, here is the idea:

- Every point in the circuit whose voltage is not already known (everything except the "0" ground reference) is treated as an unknown number to be solved for.
- The current flowing through every voltage source — including the invisible zero-volt "sense" sources from Section 3.3 — is also treated as an unknown number to be solved for.
- For every connection point, ngspice writes down a basic law of electricity: the total current flowing out of that point, through every part attached to it, must add up to exactly zero. (This law is called Kirchhoff's Current Law, and it comes up again in Section 7.) A resistor connecting two points contributes an amount of current equal to the voltage difference between those two points, divided by the resistance.
- For every voltage source, ngspice writes down the fact that the voltage difference between its two terminals must equal exactly whatever voltage that source is set to.

Put together, this produces a system of equations, and ngspice solves it using matrix math, giving back one specific number for every unknown voltage and every unknown current.

For simple DC calculations, a capacitor is treated as a complete break in the circuit (no current can flow through it, since it is "fully charged" and in equilibrium), and an inductor is treated as a plain wire with zero resistance. If the circuit contains a diode, the math becomes more complicated because diodes do not behave in a simple straight-line way, so ngspice repeats the calculation over and over, refining its guess each time (a method called Newton-Raphson iteration), until the answer stops changing.

For AC calculations (explained more in Section 9), the same basic approach is used, but every voltage and current becomes a more sophisticated kind of number called a **phasor**, which captures both a size and a timing offset (phase), and capacitors and inductors are given frequency-dependent resistance-like values instead of being treated as a simple break or a simple wire.

4.2 Reading ngspice's answer back

Once ngspice finishes, it prints its results as plain lines of text, each one looking like "name equals value." A function in the backend's main server file — responsible for parsing that output — reads through those lines and turns them into an organized internal

lookup table, matching each name (like the voltage of net "1", or the current through a particular sense source) to its numeric value.

For a DC calculation, these values are plain real numbers. But because the exact same code also has to handle AC calculations (where values are phasors, which have two parts), DC numbers are stored using the same two-part format, just with the second part always set to zero. This lets the rest of the system treat DC and AC results using identical logic afterward, without needing two separate code paths.

If ngspice's output is ever missing something that was expected — which can happen if the circuit is unsolvable (for example, if there is no path to ground somewhere, or two separate ideal batteries are directly wired in a way that contradicts each other) — this parsing step simply reports that nothing could be read for the circuit. The backend then tells the frontend that the simulation failed, and passes along ngspice's own error message so the person can see what went wrong.

5. Step 4 — Turning Raw Numbers Into Per-Part Results

At this point the backend has a big messy table of raw voltages and currents, labeled with internal names like "v(1)" or "vs1#branch." A function in the backend's main server file — responsible for organizing per-component results — turns this into one clean, consistent record for every single two-terminal component in the circuit. Each record holds:

- Which component it is (its label, like "R1," and its type, like "resistor")
- Which net its pin a and pin b are each connected to
- The voltage at pin a's net, and the voltage at pin b's net
- The voltage difference between pin a and pin b
- The current flowing through the part, measured as positive when it flows from pin a toward pin b
- Which direction that current is actually flowing (from a to b, from b to a, or no meaningful current at all)

Here is how each of those is actually worked out:

What's calculated	How	Extra notes
Voltage at pin a / pin b	Simply looked up from the net's solved voltage (or zero, if that net is the ground reference)	In a DC calculation this is a signed real value. In an AC calculation, only the size (not the direction or timing) of the oscillation is reported, so it is never negative. Any value smaller than one thousandth of a volt is rounded down to exactly zero.
Voltage difference across the part	The two voltages are subtracted from each other properly, as full phasors, before being turned into one single signed number (see 5.1)	This matters a lot in AC calculations — subtracting the two separately-reported sizes would often give the wrong answer, because sizes alone ignore timing. Any result smaller than one thousandth of a volt is rounded down to exactly zero.
Current through the part	Read directly from the appropriate sense source (for ordinary components) or from the source's own reported current (for batteries and other sources), then converted into one signed number	Any result smaller than one billionth of an amp is rounded down to exactly zero.
Direction	Positive current means "a to b." Negative current means "b to a." A current that got rounded down to zero means "no meaningful direction."	

5.1 Turning a two-part phasor into one simple signed number

As mentioned, an AC result is a phasor — a number with two parts (sometimes thought of as a size and a timing offset). But the screen only ever shows one plain signed number. A small helper routine handles this conversion consistently:

- It takes the overall size of the phasor (always a positive number on its own).
- It then decides whether to show that size as positive or negative, based on which of the phasor's two parts is the dominant one: usually the part that lines up in time with the original power source. If that part happens to be essentially zero (which happens for something like the current flowing through an ideal capacitor, where the current and voltage are perfectly out of step with each other), the other part is used instead to decide the sign.

For an ordinary DC calculation, there is no timing offset to worry about at all, so this routine simply hands back the number exactly as it already was.

5.2 A secondary, simpler summary

Alongside this detailed per-part breakdown, the backend also sends back a simpler flat list of individual values (for example, "the voltage at net 1 is this," or "the current through R1 is that"). This simpler list is mostly there as a convenient summary; the detailed per-part breakdown described above is what the visuals on screen actually use.

6. Sign Conventions — How to Read a Plus or Minus Correctly

Every single part in the circuit, including sources, is measured using the same consistent rule, known in electrical engineering as the **passive sign convention**. Stated simply:

> The voltage difference across a part is always calculated as "pin a's voltage minus pin b's voltage." The current through a part is always reported as positive when it flows from pin a toward pin b.

Here is what that actually looks like for different kinds of parts in practice:

Part	What a typical result looks like	How to read it
A resistor	Positive voltage difference, positive current	Current is flowing from the higher-voltage side to the lower-voltage side, which is the normal, expected behavior. A resistor's voltage difference and current always carry the same sign as each other, because a resistor always consumes power rather than producing it.
A source that is actively powering the circuit	Positive voltage difference, but negative current	This looks unusual at first, but it makes sense: inside the source itself, current is being pushed from its `−` terminal up to its `+` terminal (think of a battery pushing charge "uphill"). Outside the source, in the rest of the circuit, that same current flows out of the `+` terminal, through the circuit, and back into the `−` terminal. The negative sign here specifically means the source is supplying power to the circuit rather than absorbing it.
A source that is being charged by something else (for example, two batteries fighting against each other)	Positive current	Here, current is being forced backward into the source's `+` terminal, meaning the source is absorbing power rather than giving it out — similar to what happens when charging a rechargeable battery.

Part	What a typical result looks like	How to read it
A part with no current flowing through it at all (for example, a capacitor sitting in a steady DC circuit)	Current of exactly zero, no direction	

A handy sanity check. Because every part follows this same convention consistently, if every part's voltage difference is multiplied by its current and all of those results are added together across the entire circuit, the total should always come out to exactly zero. This reflects a basic physical fact: whatever power is produced by the sources in a circuit must exactly equal whatever power is consumed by everything else. This check is used internally to confirm the math came out correctly.

One more detail worth knowing: **on screen, current is always shown as a plain positive number, with an arrow indicating its direction.** A person never sees something confusing like "negative 2 milliamps" — instead they simply see "2 milliamps" together with an arrow pointing the correct way.

7. Wires in Depth: Voltage, Voltage Difference, and Current

Wires deserve their own careful explanation, because they behave a little differently from every other part in the circuit.

7.1 A wire is treated as perfect

In this app, a wire is modeled as a **perfectly ideal conductor**: zero resistance, zero inductance, zero capacitance — absolutely no electrical resistance to the flow of current whatsoever. Remember from Section 2.2 that both ends of a wire get merged into the exact same net by the union-find grouping process. Because of this, ngspice never even sees individual wires — it only ever sees nets, which are groups of points, not the physical wires connecting them.

This leads to a few consequences that are worth spelling out clearly:

What's being asked	The answer	Why
What voltage does one end of a wire have?	The voltage of the net it belongs to	

What's being asked	The answer	Why
What voltage does the other end of that same wire have?	The exact same voltage	Because both ends were merged into one single net
What is the voltage difference between the two ends of a wire?	**Always exactly zero volts, no matter what**	By Ohm's Law, voltage difference equals current multiplied by resistance. Since a wire's resistance is zero, the voltage difference is zero no matter how much current is flowing through it.
How much current flows through a wire?	It can be any amount	This is not decided by Ohm's Law at all (since that would just give zero divided by zero). Instead it is worked out using a different rule, Kirchhoff's Current Law, explained fully in Section 7.3 below.

The practical takeaway: **all of the voltage drop in any circuit happens across actual components (resistors, diodes, and so on) — never along a wire.** This matches how real textbooks teach idealized circuits, and it matches how ngspice itself works.

7.2 The voltage number shown on a wire

When the app displays a voltage number on a wire, this number comes from a small calculation inside the frontend's main canvas-drawing file, in a piece of logic dedicated to analyzing wires for display. In plain terms, it takes the voltage of whatever net is at one end of the wire, and the voltage of whatever net is at the other end, and averages the two together.

But because of what was just explained in Section 7.1 — both ends of any wire are always the exact same net — those two voltages are always identical, so the "averaging" is really just a safety fallback for one unusual edge case (if information about one end happens to be missing for some reason). In ordinary situations, this number is simply **the voltage of the one net that wire is part of.** This number only appears on screen when the wire-label display setting is set to show voltage.

7.3 The current flowing through a wire — worked out using Kirchhoff's Current Law

Here is a genuinely interesting puzzle: ngspice tells the backend the current flowing through each *component*, but it never says anything about the current flowing through each individual *wire*. If three separate wires all meet at one junction point, how much current is actually flowing through each of those three wires individually?

This puzzle is solved entirely on the frontend, by a dedicated function living in a file focused on working out current flow. It uses a fundamental electrical law called **Kirchhoff's Current Law**, which simply states: at any single point, the total current flowing in must exactly equal the total current flowing out.

Here is the step-by-step process this function follows:

Step 1 — figure out what each component pin is pushing into its wires. For every component, a small helper function decides how much current that component is injecting into (or pulling out of) the wires touching each of its two pins:

Pin	What it contributes to its wire
Pin a	The negative of the component's through-current (because current flowing "a to b" effectively *enters* the component at pin a, pulling current in from the wire)
Pin b	The component's through-current exactly as is (because that current *leaves* the component at pin b, pushing current out into the wire)

For example: a resistor with 2 milliamps flowing through it from pin a to pin b is effectively **pulling** 2 milliamps out of the wire touching pin a, and **pushing** 2 milliamps into the wire touching pin b.

Step 2 — build a small map, one per net. All the pins belonging to one net, along with all the wires connecting them, form a little map of their own. Each net gets handled completely separately from every other net.

Step 3 — find a simple path through that map. Starting from any one pin in the net, the function walks outward through the wires (a method called breadth-first search) and builds what is called a "spanning tree" — essentially the simplest possible set of

connections that still reaches every pin in that net, with no leftover loops. Along the way, it remembers, for every pin, exactly which single wire connects it back toward the starting point.

Step 4 — handle unknown pins. Occasionally, a pin belongs to something that is not actually a measurable component at all — the most common example being a ground symbol, which (as explained in Section 2.3) is never treated as a real device and therefore never got a current reading from ngspice at all. Such a pin is called "unknown." Kirchhoff's Current Law says that all the currents meeting at a point must balance out to zero, so:

- If there is exactly one unknown pin in a net, its contribution can be figured out by making everything else balance to zero. This is precisely how a return current flowing into a ground symbol becomes visible on the wire leading to it.
- If there is more than one unknown pin in the same net (for example, two separate ground symbols that happen to be wired together), there is no way to know how the current should split between them, so the wires in that particular net are simply left without any current value at all.

Step 5 — work backward from the outer branches inward. Walking back through the simple path built in Step 3, starting from the outermost branches and moving inward, the function adds up how much current flows up each wire toward the center: the current flowing up a particular wire equals whatever that pin itself contributes, plus the combined total of everything flowing up from all of its own branches further out. This running total is then compared against the wire's own stored direction (its defined "from" end and "to" end) to decide whether the current should be reported as flowing one way or the other along that specific wire.

Step 6 — handle closed loops made only of wire. Occasionally, two wires form a complete loop using nothing but wire — for example, two separate wires both directly connecting the exact same two pins. Since wires have zero resistance, there is no way to determine how current should split between two equally-easy paths, so any such extra "loop-closing" wire is simply reported as carrying zero current.

Why this approach actually works

In a simple path with no loops, there is only ever one possible route between any two pins. This means that cutting any single wire in that simple path always splits the whole net cleanly into exactly two separate groups, and Kirchhoff's Current Law then forces that wire to carry precisely the total current belonging to one of those two groups. This one

idea correctly handles every kind of wiring pattern someone might draw: T-shaped junctions, star-shaped junctions with many wires meeting at one point, long chains of wires, and more.

A simple example: current splitting at a T-junction

Imagine a battery's `+` terminal connected by one wire to a resistor's first pin, and that same resistor's first pin is also connected by a second wire to a second resistor's first pin. Suppose the battery is pushing out 3 milliamps total, the first resistor is drawing 1 milliamp, and the second resistor is drawing 2 milliamps.

There is no separate "junction" object anywhere in this system — wires always connect exactly two pins, so the junction here is simply the first resistor's pin, which happens to have two different wires touching it. Working through the steps above, the wire leaving the battery is found to carry all 3 milliamps, and the wire leading to the second resistor carries 2 of those milliamps, while the first resistor keeps 1 milliamp for itself. That checks out perfectly with Kirchhoff's Current Law: 3 milliamps in equals 1 milliamp plus 2 milliamps out.

8. What Actually Gets Drawn on Screen

None of these visuals appear until a simulation has fully finished and the per-part results are ready. Here is a plain breakdown of every visual element and exactly what feeds it:

What you see	What produces it	How it behaves
Small voltage labels ("pills") floating near connection points	A piece of overlay-drawing logic in the frontend, using a helper that builds these labels from the per-part results	One label appears per distinct net and position. Ground points, or any voltage smaller than one thousandth of a volt, are shown as a plain "0.00 V."
A small badge on a component showing its voltage difference and current	The component-drawing logic, pulling straight from that component's own record in the per-part results	Can be shown either for components someone has specifically selected, or for every component at once, depending on a toggle.

What you see	What produces it	How it behaves
The full results table	A dedicated results-table panel, built directly from the full per-part results list	Shows the voltage difference (with its sign), the size of the current (always positive), and the direction, written as "from this point to that point."
Moving dashes and an arrow along a wire	The wire-drawing logic, fed by the wire-current calculation described in Section 7.3	The arrow sits at the middle of the longest straight stretch of the wire and points in the direction current is flowing. The dashes animate faster when more current is flowing (very fast for large currents, slower for smaller ones), and the whole animation disappears entirely for current so small it is essentially zero. This can be turned on or off with a toggle.
Small arrows and dots near the two ends of a wire showing voltage direction	A helper function that figures out each pin's relative polarity, based on the per-part results	For whichever component sits at each end of the wire: if that end touches the component's higher-voltage pin, the arrow points inward (showing voltage "dropping into" that component); if it touches the lower-voltage pin, the arrow points outward. If the two ends disagree with each other, each half of the wire animates independently toward its own component. A component with a voltage difference smaller than one thousandth of a volt gets no arrow at all. This can also be turned on or off with a toggle.

What you see	What produces it	How it behaves
A small label on a wire showing its current and/or its voltage	The same wire-analysis logic mentioned in Section 7.2	Shows the size of the current and/or the net's voltage, depending on a wire-label display setting. Whichever wire is currently selected always shows its current value regardless of that setting.

One subtlety worth noting clearly: **the little voltage arrows near the ends of a wire describe the components sitting at each end of that wire — they do not describe any voltage drop happening along the wire itself.** As explained in Section 7.1, the voltage drop along any wire is always exactly zero, no matter what those end-arrows show.

9. DC vs AC Analysis, Explained Simply

The backend's main simulation route decides between two different kinds of analysis using a simple rule: **if the circuit contains at least one AC source, and the person has not explicitly asked for a plain DC calculation, an AC steady-state calculation is run.** Otherwise, a plain DC operating-point calculation is run.

Here is what each one actually means, and how they differ:

	DC calculation	AC calculation
How a capacitor behaves	Treated as a complete break in the circuit	Treated as having a frequency-dependent resistance-like value
How an inductor behaves	Treated as a plain wire with no resistance	Treated as having a frequency-dependent resistance-like value
How DC sources behave	Contribute their actual voltage	Dropped entirely — treated as contributing nothing, and the results message explicitly notes that DC sources were not included

	DC calculation	AC calculation
How AC sources behave	Contribute zero (they are turned off for the DC calculation)	Contribute their oscillating amplitude, at their own timing
What the reported numbers mean	Exact, signed, real values	**Peak amplitudes** (the maximum size the oscillation reaches), not signed values
Voltage at each net	Can be positive or negative	Always reported as a plain non-negative size
Sign of voltage difference and current	The true real-world sign	A sign chosen relative to the timing of the original source, as explained in Section 5.1

If a circuit happens to contain several different AC sources running at different frequencies, the calculation is only performed at the frequency of the very first AC source found, and the results message notes this clearly so nobody is misled.

One more subtlety worth calling out: in an AC calculation, **the voltage difference across a part is usually not simply "voltage at one end minus voltage at the other end."** The voltages shown at each individual point are only sizes (with no timing information), but the voltage difference across a part is calculated properly from the full phasors, which do carry timing information. As a concrete example, the voltage across a capacitor in a simple circuit with one resistor and one capacitor is generally *not* just the plain difference between the two reported point-voltages — the timing offset genuinely matters here.

10. A Complete Worked Example

To tie everything together, here is one full circuit walked through every stage described above.

The circuit: a 10-volt battery, connected through a 1,000-ohm resistor and then a 4,000-ohm resistor wired one after the other (in series), with a ground symbol placed at the battery's `-` terminal.

Stage 1 — building the connection list

The union-find grouping (Section 2.2) finds that the ground symbol's pin, together with the battery's `-` pin, all belong to the net named `"0"`. The battery's `+` pin and the first resistor's first pin both belong to net `"1"`. The first resistor's second pin and the second resistor's first pin both belong to net `"2"`. The second resistor's second pin connects back into net `"0"` as well.

So the connection list says: the battery sits between net 1 and net 0; the first resistor sits between net 1 and net 2; the second resistor sits between net 2 and net 0.

Stage 2 — building the solver file

The backend writes out a deck containing the battery (10 volts, between nets 1 and 0), the first resistor (1,000 ohms, with an invisible zero-volt sense source right after it), and the second resistor (4,000 ohms, with its own invisible sense source). The bottom of the deck asks ngspice to solve the circuit and print the voltage at net 1, the voltage at net 2, the battery's current, and the current through each of the two sense sources.

Stage 3 — ngspice solves it

Using simple series-circuit math (which is exactly what Modified Nodal Analysis works out automatically): the total resistance is 1,000 plus 4,000, which is 5,000 ohms. The current flowing is 10 volts divided by 5,000 ohms, which is 0.002 amps, or 2 milliamps. Net 1 sits at the full battery voltage, 10 volts. Net 2 sits at 10 volts minus the drop across the first resistor (2 milliamps times 1,000 ohms, which is 2 volts), leaving 8 volts.

Stage 4 — organizing the per-part results

After the backend organizes these raw numbers into clean per-part records:

Part	From → to	Voltage at pin a	Voltage at pin b	Voltage difference	Current	Direction
Battery	net 1 → net 0	10 V	0 V	+10 V	−2 mA	b to a (meaning: it is actively supplying power)
First resistor	net 1 → net 2	10 V	8 V	+2 V	+2 mA	a to b
Second resistor	net 2 → net 0	8 V	0 V	+8 V	+2 mA	a to b

As a sanity check (Section 6): 10 times negative 0.002, plus 2 times 0.002, plus 8 times 0.002, works out to negative 0.02 plus 0.004 plus 0.016, which equals exactly zero — confirming the power supplied by the battery exactly equals the power consumed by the two resistors combined.

Stage 5 — working out the wires

There are four wires in this circuit: one from the battery's `+` to the first resistor, one from the first resistor to the second resistor, one from the second resistor down to the ground symbol, and one from the ground symbol back to the battery's `−`. Every single one of these wires carries the full 2 milliamps, flowing in the natural direction around the loop, and every single one of them shows a voltage difference of exactly zero along its own length — all of the actual voltage drop happens across the two resistors, never along any wire.

The key lesson from this example: the full 10 volts the battery provides is dropped 2 volts across the first resistor and 8 volts across the second resistor — adding up to all 10 volts — and absolutely none of it is lost or dropped anywhere along the connecting wires themselves.

11. Known Limitations, Explained Plainly

No system is perfect, and it is worth understanding what this one does not do, so results are never misread:

- **Transient (over-time) simulation is not actually performed.** The app's interface offers a "Transient" mode along with settings for simulation step size and stop time, but behind the scenes, the backend only ever performs either a single DC snapshot calculation or a single-frequency AC calculation. Any transient or frequency-sweep settings someone configures are simply accepted and then quietly ignored.
- **Wires are perfectly ideal**, as explained thoroughly in Section 7. There is no such thing as wire resistance in this model, so the voltage drop along any wire is always precisely zero, and a wire cannot be used to represent a real physical cable with any resistance of its own. To model something like that, a small resistor should be placed in the circuit instead.
- **Loops made entirely of wire** (for instance, two separate wires both connecting the very same two pins) will show zero current on whichever wire closes the loop, simply because there is no way to determine how the current should split between two equally valid, zero-resistance paths.

- **A net with more than one "unknown" pin** (for example, two separate ground symbols that end up wired into the very same net) leaves the wire currents in that net undetermined, for the same underlying reason described in Section 7.3.
- **Dependent sources are simplified.** A dependent source (one whose output is supposed to depend on some other part of the circuit) is currently simulated as if it were simply a plain, independent, fixed voltage source set to its own voltage value. The actual controlling relationship it is supposed to have with the rest of the circuit is not captured or honored.
- **DC and AC sources cannot currently work together properly.** When an AC calculation is performed, any DC sources present are dropped out entirely, so there is no way to see the combined effect of a steady DC bias layered underneath an AC signal (something real engineers call superposition).
- **AC voltage readings at individual points lose their sign and timing information**, since only their size is reported, as explained in Section 9.
- **Certain circuits simply cannot be solved at all** — for example, a point with absolutely no path back to ground, or a closed loop made up only of ideal voltage sources contradicting each other. In these cases, ngspice itself fails, and the person is shown that the simulation did not succeed, along with ngspice's own error details.
- **Only two-terminal parts are fully supported.** Anything with fewer than two connection points is skipped entirely when building the solver file, and only the first two connection points of any part are ever used, even if a part happened to have more.